



Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Segundo Ciclo 2026

Entrega 2 - Proyecto POO

Ingeniería en Ciencias de la computación
y tecnología de la información

2026

Índice

Identificación de requisitos funcionales	3
Identificación y Descripción de Clases Necesarias	7
Model	7
Componente	7
LED (Hereda de Componente)	7
Resistencia (Hereda de Componente)	7
CompuertaLogica (Hereda de Componente)	7
CompuertaAND (Hereda de CompuertaLogica)	7
CompuertaOR (Hereda de CompuertaLogica)	8
Switch (Hereda de Componente)	8
Minijuego	8
ProgresoJugador	8
GestorPersistenciaJSON	8
Controller	8
ControladorNivel (Extiende InputAdapter)	8
View	9
VistaNivel (Extiende ScreenAdapter)	9
VistaMenu (Extiende ScreenAdapter)	9
JuegoPrincipal (Extiende Game)	9
Constants	9
Main	9
Diseño del Sistema / UML	10
Investigación de la tecnología disponible	11
Investigación de la tecnología disponible para persistencia de datos	12
¿Por qué JSON es la opción más óptima para nuestro juego?	13
Planificación y gestión:	13
Repositorio GitHub	15
Referencias	18

Identificación de requisitos funcionales

Módulo	Requisito Funcional	Interacción	Prioridad
Juego (Gameplay)	Arrastre de Componentes (<i>Drag & Drop</i>)	El usuario hace clic y arrastra componentes (LED, resistencias, compuertas) hacia los nodos del circuito.	1
Juego (Gameplay)	Evaluación y Lógica del Circuito	El motor evalúa automáticamente si la conexión cumple la condición física/lógica (polaridad, valor óhmico o compuerta AND).	1
Juego (Gameplay)	Temporizador Contrarreloj	Muestra la cuenta regresiva de 5 a 8 segundos; al agotarse el tiempo sin resolver el puzzle, declara el nivel como fallido.	1
Juego (Gameplay)	Conmutación y Rotación de Piezas	El usuario hace clic sobre un switch para alternar entre 0 y 1, o sobre el LED para invertir su sentido de polaridad.	2

Juego (Gameplay)	Reacción Visual de Sprites	La pieza cambia de expresión en tiempo real según el resultado: neutra (duda), feliz (éxito) o quemada (fallo).	2
Menú Principal	Iniciar Partida y Selector de Niveles	El usuario interactúa con la pantalla de inicio para empezar a jugar o elegir un nivel de circuito específico.	2
Sistema / Datos	Persistencia de Progreso en json	El sistema guarda y carga de forma persistente los niveles desbloqueado en un archivo JSON	2
Juego (Gameplay)	Pausa y Reinicio Rápido	El usuario puede pausar el temporizador para reiniciar el puzzle o regresar al menú principal.	2
Menú Principal	Tutorial e Instrucciones	El usuario consulta las reglas rápidas del juego, mecánicas de arrastre y significados de los estados visuales.	3

Configuración	Panel de Sonido y Audio	El usuario puede silenciar o ajustar el volumen de los efectos de acierto/fallo y música de fondo.	3
----------------------	-------------------------	--	----------

Prioridad 1: Lo indispensable.

Prioridad 2: funciones importantes de flujo, control y guardado..

Prioridad 3: Detalles estéticos y de configuración.

Identificación y Descripción de Clases Necesarias

Model

Componente

Clase base abstracta de la cual heredan todas las piezas del circuito. Almacena las coordenadas espaciales en pantalla, un identificador único de la pieza y su estado actual para gestionar las representaciones de interfaz.

- **Atributos:** identificador (String), posicionX (float), posicionY (float), estadoActual (String: "NEUTRO", "EXITO", "ERROR").
- **Métodos:** evaluarEstado() [abstracto], actualizarPosicion(nuevaX, nuevaY), Getters, Setters, toString().

LED (Hereda de Componente)

Modela un diodo LED interactivo. Valida que la polaridad eléctrica de la conexión establecida por el usuario sea la adecuada dentro del circuito.

- **Atributos:** polaridadCorrecta (boolean).
- **Métodos:** evaluarEstado() [sobrescrito], Getters, Setters, toString().

Resistencia (Hereda de Componente)

Representa un resistor que evita sobrecargas. Verifica que el valor en ohmios asignado por el jugador coincida con la especificación requerida por el nivel.

- **Atributos:** valorOhmios (int), valorRequerido (int).
- **Métodos:** evaluarEstado() [sobrescrito], Getters, Setters, toString().

CompuertaLogica (Hereda de Componente)

Clase abstracta intermedia para compuertas digitales de dos entradas. Recibe señales booleanas de entrada y procesa el resultado binario de salida.

- **Atributos:** entradaA (boolean), entradaB (boolean), salida (boolean).
- **Métodos:** setEntradas(a, b), calcularSalida() [abstracto], evaluarEstado() [sobrescrito], Getters, Setters, toString().

CompuertaAND (Hereda de CompuertaLogica)

Modela una compuerta lógica AND de dos entradas digitales.

- **Métodos:** CompuertaAND(identificador, posicionX, posicionY, entradaA, entradaB), calcularSalida() [sobrescrito: entradaA & entradaB], toString().

CompuertaOR (Hereda de CompuertaLogica)

Modela una compuerta lógica OR de dos entradas digitales.

- **Métodos:** CompuertaOR(identificador, posicionX, posicionY), calcularSalida() [sobrescrito: entradaA || entradaB], toString().

Switch (Hereda de Componente)

Modela un interruptor que conmuta el paso de corriente eléctrica al interactuar con él.

- **Atributos:** cerrado (boolean).
- **Métodos:** Switch(identificador, posicionX, posicionY, cerradoInicial), conmutar(), evaluarEstado() [sobrescrito], Getters, Setters, toString().

Minijuego

Gestiona el flujo de ejecución del nivel, la lógica del temporizador regresivo y la colección de componentes pertenecientes al circuito.

- **Atributos:** numeroNivel (int), tituloReto (String), tiempoLimite (float), tiempoRestante (float), completado (boolean), ganado (boolean), componentes (List<Componente>).
- **Métodos:** Minijuego(numeroNivel, tituloReto, tiempoLimite), actualizar(deltaTiempo), agregarComponente(componente), evaluarCircuitoCompleto(), Getters, Setters, toString().

ProgresoJugador

Contenedor de datos encargado de estructurar la información del avance registrado por el usuario.

- **Atributos:** nivelMaximoDesbloqueado (int).
- **Métodos:** Constructor, Getters, Setters, toString().

GestorPersistenciaJSON

Maneja la lectura y escritura en disco del archivo local de guardado en formato JSON.

- **Métodos:** guardarProgreso(progreso, ruta), cargarProgreso(ruta).

Controller

ControladorNivel (Extiende InputAdapter)

Captura y procesa los eventos del cursor e interacción del usuario para manipular las piezas del circuito.

- **Atributos:** minijuego (Minijuego), camara (OrthographicCamera), componenteSeleccionado (Componente).
- **Métodos:** ControladorNivel(minijuego, camara), touchDown(screenX, screenY, pointer, button), touchDragged(screenX, screenY, pointer), touchUp(screenX, screenY, pointer, button), Getters, Setters, toString().

View

VistaNivel (Extiende ScreenAdapter)

Renderiza el escenario de juego leyendo las posiciones y estados de las piezas directamente del modelo.

- **Atributos:** minijuego (Minijuego), batch (SpriteBatch), camara (OrthographicCamera), mapaTexturas (Map<String, Texture>).
- **Métodos:** VistaNivel(minijuego), render(delta), resize(width, height), dispose().

VistaMenu (Extiende ScreenAdapter)

Despliega la interfaz del menú principal y sus opciones de navegación.

- **Métodos:** render(delta), show(), dispose(), toString().

JuegoPrincipal (Extiende Game)

Clase central encargada de gestionar el cambio y transición entre pantallas del juego.

- **Métodos:** create(), setScreen(), dispose().

Constants

Contenedor de variables globales estáticas para parametrizar valores fijos del sistema.

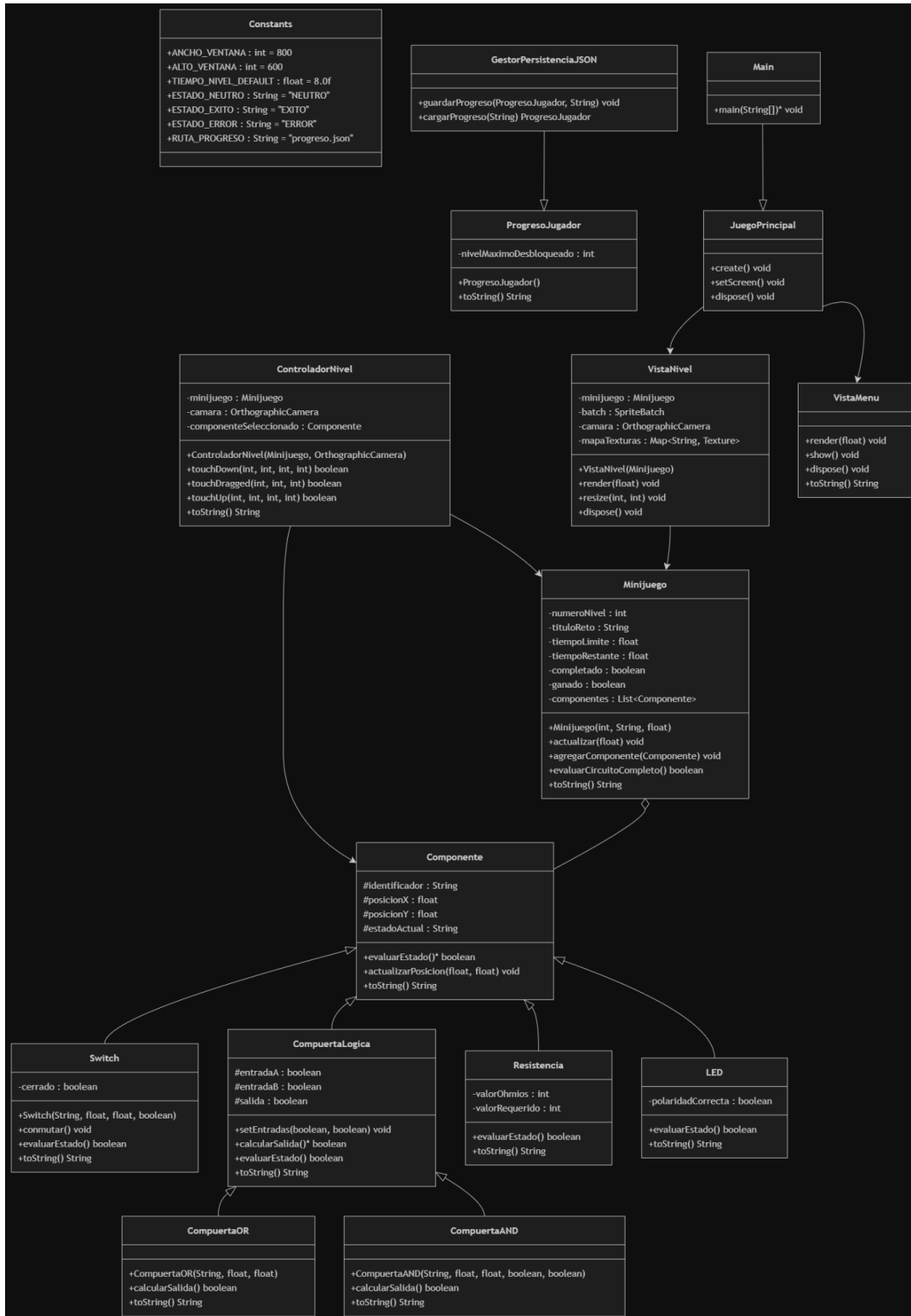
- **Atributos estáticos:** ANCHO_VENTANA = 800, ALTO_VENTANA = 600, TIEMPO_NIVEL_DEFAULT = 8.0f, ESTADO_NEUTRO = "NEUTRO", ESTADO_EXITO = "EXITO", ESTADO_ERROR = "ERROR", RUTA_PROGRESO = "progreso.json".

Main

Punto de entrada para la ejecución de la aplicación en escritorio.

- **Métodos:** main(args) [configura resolución 800x600 a 60 FPS con Lwjgl3Application].

Diseño del Sistema / UML



(Está en el Repositorio con mejor calidad)

Investigación de la tecnología disponible

¿Qué es LibGDX?

LibGDX es un framework multiplataforma de desarrollo de videojuegos en Java basado en OpenGL (ES), fue creado y pensado para Windows, Linux, macOS, Android, navegadores web e iOS. A diferencia de motores con editor visual como unity, no obliga al usuario a utilizar un estilo de diseño o codificación específico por lo que deja una total libertad para estructurar el juego como se prefiera por eso encaja bien con el patrón arquitectural MVC hecha a mano como la definida para este proyecto.

Arquitectura interna

LibGDX, no es una “caja negra” a diferencia de linux, donde no se sabe que pasa por dentro, por lo que es Java puro y se reduce a cinco interfaces (Application, Files, Input, Net, Audio, Graphics) que cada backend implementa para el sistema operativo correspondiente. El único código específico de cada plataforma que hay que escribir es una "clase de arranque", que instancia la implementación concreta de Application para esa plataforma, en nuestro caso, la clase Main. Todo lo demás: Componente, LED, Minijuego, ControladorNivel, etc. es Java estándar.

El ciclo de vida del juego se controla mediante la interfaz ApplicationListener (o su clase base ApplicationAdapter), con métodos create(), render(), resize(), pause(), resume() y dispose() que el framework invoca automáticamente. Esto se traduce directamente en el diseño de clases: VistaNivel extiende ScreenAdapter e implementa render(delta), resize(width, height) y dispose(), mientras que JuegoPrincipal extiende Game y actúa como el ApplicationListener raíz que delega en la Screen activa mediante setScreen().

Por qué es adecuado para este proyecto:

- **Ciclo de desarrollo rápido en escritorio:** el framework permite ejecutar y depurar el juego directamente en el escritorio en vez de desplegar a Android/iOS/JavaScript después de cada cambio.
- **Todo lo necesario ya incluido:** provee todo lo requerido para desarrollar juegos 2D y 3D multiplataforma con Java, incluyendo SpriteBatch, OrthographicCamera, manejo de input (touchDown/touchDragged/touchUp, usados en ControladorNivel), gestión de texturas y el ciclo Game - Screen reflejado en JuegoPrincipal, VistaNivel y VistaMenu.
- **Costo y comunidad:** se distribuye bajo licencia Apache 2.0, por lo que es gratis y también cuenta con numerosas herramientas y librerías

Comparación con otras alternativas en Java:

- **JavaFX:** está enfocado en gráficos y GUI de escritorio, con toda clase de controles de interfaz, scene graph y estilizado por CSS, pensado más para aplicaciones visuales y formularios que para videojuegos. En cambio, LibGDX, está diseñado específicamente para juegos, con sprite batching, atlas de texturas y soporte de motores de física, las cuales son funcionalidades que el proyecto necesita de forma nativa.
- **Swing/AWT:** no está pensado para bucles de render a 60 FPS ni para gestión eficiente de texturas/OpenGL, por lo que no es una buena alternativa para un juego con temporizador y animaciones en tiempo real.

Investigación de la tecnología disponible para persistencia de datos

La persistencia de datos en videojuegos permite guardar el progreso, niveles y configuración para recuperarlos después. En un juego de puzzle de circuitos con estados de sprites feliz/neutro/quemado, se requiere guardar objetos complejos del modelo: Circuito, Componente, LED, CompuertaAND, posiciones y estado de evaluación.

LibGDX ofrece varias tecnologías nativas para esto sin necesidad de librerías externas.

Tecnologías disponibles en LibGDX

1. Preferences

Es un sistema tipo HashMap que guarda claves y valores primitivos. LibGDX lo describe como una forma simple de almacenar datos pequeños como configuración de usuario o partidas pequeñas (LibGDX, n.d.-b). Es ideal para volumen, sonido o último nivel desbloqueado, pero no para grafos de objetos complejos. Además es la única opción para persistencia en HTML5.

2. JSON - com.badlogic.gdx.utils.Json

LibGDX puede realizar serialización automática de objetos a JSON y deserialización de JSON a objetos. La API está compuesta por 4 clases: JsonWriter, JsonReader, JsonValue y Json (LibGDX, n.d.-a). Json usa reflexión para serializar automáticamente POJOs, Array, OrderedMap, String, Float y Boolean. Soporta formatos minimal, JavaScript y JSON estándar (LibGDX, n.d.-a).

Otras alternativas descartadas:

1. **XML:** Más verboso, parsing más pesado y menos legible que JSON. LibGDX lo soporta pero no aporta ventajas para un puzzle.
2. **Binario / Serializable de Java:** No es legible por humanos, dificulta el debug de niveles y tiene problemas de versionado.

3. **SQLite**: Requiere driver y sobrecarga de SQL para un juego que solo necesita guardar archivos de nivel de 10-50 KB.

¿Por qué JSON es la opción más óptima para nuestro juego?

JSON es utilizable de forma nativa en LibGDX a través de `com.badlogic.gdx.utils.Json` y `Gdx.files.local`. Es la opción óptima frente a Preferences, XML y binario para un puzzle de lógica porque combina serialización automática de objetos del modelo POO, legibilidad para pruebas unitarias con JUnit (CircuitoTest) y edición de niveles, peso mínimo y compatibilidad total con el stack Java puro que exige el curso.

Planificación y gestión:

Tarea	Descripción de la Tarea	Horas estimadas	Responsable	Fecha de terminación
Definición de Arquitectura Base	Análisis y creación de la versión simple del programa con clases abstractas y modelos iniciales.	5 horas	Justin B. / Daniel V.	30/08/2026
Punto de Entrada y Setup	Análisis de recursos, creación de la clase <code>Main.java</code> e inicialización del repositorio.	3 horas	Julio Ortiz	30/08/2026
Diseño de Interfaz y Wireframes	Diseño de mockups, wireframes y planteamiento visual del prototipo.	4 horas	Daniel Villatoro	30/08/2026
Requisitos y Variables Globales	Redacción de la lista priorizada de requisitos y estructuración del archivo <code>Constantes.java</code> .	4 horas	Marcos Ambelis	30/08/2026
Programación Lógica y Persistencia	Implementación de clases (Componente, LED, etc.), método <code>evaluarEstado()</code> y estructura <code>.json</code> .	10 horas	Justin Barrios	06/09/2026

Programación Interfaz y Sprites	Dibujo/exportación de sprites 2D e implementación de VistaNivel.java usando SpriteBatch.	12 horas	Daniel Villatoro	08/09/2026
Programación Eventos e Inputs	Implementación de ControladorNivel.java, captura de eventos del mouse y proyección de coordenadas.	9 horas	Marcos Ambelis	08/09/2026
Automatización Pruebas Unitarias	Construcción de pruebas con JUnit (CircuitoTest.java) para el Modelo sin depender de la GUI.	7 horas	Julio Ortiz	09/09/2026
Pruebas de Integración y Bitácora	Ejecución de pruebas manuales, detección de fallos visuales y registro de incidencias.	6 horas	Julio Ortiz	13/09/2026
Pruebas con Usuarios Finales	Muestra de la aplicación integrada a usuarios y recolección de retroalimentación.	4 horas	Daniel V. / Julio O.	15/09/2026
Diagrama UML y Justificación	Elaboración del diagrama UML y redacción de la investigación de persistencia de datos.	6 horas	Marcos Ambelis	18/09/2026
Ensamblaje y Revisión GitHub	Unificación del documento PDF final y verificación del PR en GitHub hacia main.	4 horas	Justin / Marcos A.	20/09/2026

Repositorio GitHub

[visamonks/JavaPOO](#) Proyecto: Videojuego en Java que enseña diagramas esquemáticos y circuitos mediante acertijos rápidos contrarreloj. Los componentes son personajes 2D expresivos que dan feedback inmediato, validados en tiempo real bajo arquitectura MVC.

Referencias

- Ecma International. (2017). *ECMA-404 The JSON data interchange syntax* (2.^a ed.). <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>
- JSON.org. (s. f.). *Introducing JSON*. Recuperado el 4 de septiembre de 2026, de <https://www.json.org/json-en.html>
- LibGDX. (s. f.-a). *File handling*. libGDX Wiki. Recuperado el 4 de septiembre de 2026, de <https://libgdx.com/wiki/file-handling>
- LibGDX. (s. f.-b). *Preferences*. libGDX Wiki. Recuperado el 4 de septiembre de 2026, de <https://libgdx.com/wiki/preferences>
- LibGDX. (s. f.-c). *Reading and writing JSON*. libGDX Wiki. Recuperado el 4 de septiembre de 2026, de <https://libgdx.com/wiki/utls/reading-and-writing-json>
- LibGDX. (s. f.-d). *The life cycle*. libGDX Wiki. Recuperado el 4 de septiembre de 2026, de <https://libgdx.com/wiki/app/the-life-cycle>
- LibGDX Contributors. (s. f.-a). *Goals & features* [Wiki en GitHub]. GitHub. Recuperado el 4 de septiembre de 2026, de <https://github.com/sinistersnare/libgdx/wiki/Goals-%26-Features>
- LibGDX Contributors. (s. f.-b). *libGDX* [Repositorio de código]. GitHub. Recuperado el 4 de septiembre de 2026, de <https://github.com/libGDX/libGDX>
- LibGDX Contributors. (s. f.-c). *Reading and writing JSON* [Wiki en GitHub]. GitHub. Recuperado el 4 de septiembre de 2026, de <https://github.com/libgdx/libgdx.github.io/blob/HEAD/wiki/utls/reading-and-writing-json.md>
- LibGDX Contributors. (s. f.-d). *Starter classes & configuration* [Wiki en GitHub]. GitHub. Recuperado el 4 de septiembre de 2026, de <https://github.com/sinistersnare/libgdx/wiki/Starter-classes-%26-configuration>
- LibGDX Contributors. (s. f.-e). *The application framework* [Wiki en GitHub]. GitHub. Recuperado el 4 de septiembre de 2026, de <https://github.com/sinistersnare/libgdx/wiki/The-Application-Framework>
- Sagristà, T. (2013, 14 de noviembre). *From Slick to Libgdx*. Toni Sagristà Blog. <https://tonisagrista.com/blog/2013/slick-libgdx>
- StackShare. (s. f.). *JavaFX vs libGDX*. Recuperado el 4 de septiembre de 2026, de <https://stackshare.io/stackups/javafx-vs-libgdx>